



7-Day Code Masterclass: CDM Library System

Daily Codebase Curriculum for Capstone Defense Preparation · Colegio de Montalban



DAY 1: Database Setup & Schema

File: `server/db.js`

- **Foreign Key Enforcement:** Enables `db.pragma('foreign_keys = ON')` to prevent orphan transactions.
- **Schema Definitions:** Creates 5 core tables (`librarians`, `books`, `transactions`, `reservations`, `students`).
- **Auto-Seeding:** Runs count checks and populates initial default admin and book records automatically.

Q: Where are database relationships enforced?

A: In `server/db.js` line 17 and inside SQL table definitions via `FOREIGN KEY ... REFERENCES`.



DAY 2: Express Server Setup & Rate Limiter

File: `server/server.js`

- **Express Initialization:** Configures server port 5002, CORS, and JSON parsing middleware.
- **Rate Limiting Engine:** `createRateLimiter` tracks client IPs in an in-memory Map.
- **Limits:** `apiLimiter` (max 300 reqs/15m) and `authLimiter` (max 15 attempts/15m).

Q: How does your server block brute-force password guessing?

A: Custom `createRateLimiter` middleware tracks client IPs and returns HTTP 429 after 15 auth attempts in 15 mins.



DAY 3: Authentication & Security Engine

File: `server/server.js`

- **PBKDF2 Hashing:** Uses `crypto.pbkdf2Sync` with 16-byte random salt and SHA-512 key derivation.
- **Timing Attack Protection:** Verifies password hashes safely using `crypto.timingSafeEqual`.
- **JWT Session Handler:** Generates 24h JWT tokens and validates them via `authenticateToken` middleware.

Q: How do you protect passwords and verify user sessions?

A: Passwords are hashed with PBKDF2 SHA-512 + salt. Sessions are validated via 24-hour JWT tokens.

DAY 4: Book Catalog & Inventory Logic

File: `server/server.js`

- **Catalog CRUD:** Handles GET, POST, PUT, DELETE routes for book metadata.
- **ISBN Uniqueness:** Verifies ISBN uniqueness before creating or updating titles.
- **Stock Auto-Recalculation:** `newAvailable = Math.max(0, total - copiesBorrowed)` upon edit.

Q: What happens if a librarian updates total copies of a borrowed book?

A: The server calculates `copiesBorrowed = total - available` and preserves active loan counts automatically.

DAY 5: Loans, Overdue Engine & Penalties

File: `server/server.js`

- **Auto Overdue Detection:** Runs `UPDATE transactions SET status = 'overdue' WHERE due_date < today` on fetch.
- **Return Processing:** Evaluates book condition (Good, Damaged, Lost) and computes late penalty fines dynamically.

Q: How does the system calculate overdue fines?

A: Express compares `due_date` against current/return timestamps, calculates late days, and computes penalty fees.

DAY 6: Frontend React Architecture

File: `src/app/App.tsx`

- **React State & Hooks:** Uses `useState` for UI views, modals, and tables; `useEffect` for initial API fetch.
- **Auth Header:** Attaches `Authorization: Bearer <token>` header to all API requests.

Q: How does React communicate securely with Express?

A: React stores the JWT token in state and attaches it to HTTP Authorization headers on fetch calls.

DAY 7: Public Promotional Web Portal

Files: `promotional-website/app.js` & `index.html`

- **Zero-Login Live Catalog:** Sends public fetch requests to `/api/books` to display live book search preview.
- **Direct APK Download:** Serves student mobile app installer package via download badge.

Q: Can guests search the library catalog without logging in?

A: Yes! The promotional portal queries the public `/api/books` REST endpoint dynamically.